POS TECH

TECH CHALLENGE

Tech Challenge é o projeto que englobará os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, em princípio, deve ser desenvolvida em grupo. Importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale **90%** da nota de todas as disciplinas da fase.

Nota Importante sobre o Ambiente de Nuvem (Terraform & IAM)

Assim como na fase anterior, temos restrições para quem usa o AWS Academy:

Opção A: AWS Academy:

- Seu código Terraform **não pode criar Roles ou Policies de IAM**.
- O Terraform deve ser configurado para utilizar a **LabRole** existente para o Cluster EKS e para os Node Groups.
- Você deverá importar os dados da LabRole (via data source no Terraform ou variável) para associá-la aos recursos.

Opção B: Conta Pessoal:

- Você tem liberdade total para criar as roles de IAM via Terraform (recomendado para um portfólio profissional).

DESAFIO

A **DevOps Solutions Inc.** aprovou a arquitetura de microsserviços da Fase 2. No entanto, a operação se tornou insustentável.

- Os desenvolvedores estão rodando kubectl apply de suas máquinas locais, gerando conflitos de versão.
- As credenciais do banco de dados estão sendo passadas em arquivos de texto sem segurança.
- Recentemente, uma vulnerabilidade em uma biblioteca Go passou despercebida e foi para produção.

- Recriar o ambiente de homologação leva dias porque foi feito manualmente no console.

A ordem agora é: "Se não está no código, não existe". Precisamos de infraestrutura imutável, pipelines de segurança (DevSecOps) e GitOps para o deploy.

Você deve automatizar **toda** a infraestrutura e o ciclo de vida dos 5 microsserviços do ToggleMaster (auth, flag, targeting, evaluation, analytics) utilizando as práticas de IaC, CI/CD e DevSecOps.

REQUISITOS TÉCNICOS

1. Infraestrutura como Código (Terraform)

Você deve substituir a criação manual da Fase 2 por código Terraform. Crie um projeto Terraform organizado (preferencialmente usando módulos) que provisione:

1. **Networking:** VPC, Subnets (Públicas e Privadas), Internet Gateway e Route Tables.
2. **Cluster EKS:** O cluster Kubernetes e seus Node Groups.
   - *Atenção Academy:* Lembre-se de associar a LabRole.
3. **Bancos de Dados:**
   - **3** instâncias RDS (PostgreSQL).
   - **1** Cluster ElastiCache (Redis).
   - **1** Tabela DynamoDB (ToggleMasterAnalytics).
4. **Mensageria:** 1 Fila SQS.
5. **Repositórios:** 5 repositórios no ECR (opcional via Terraform, mas recomendado).

Requisito de Estado: O terraform.tfstate não pode ficar local. Configure o Backend Remoto usando um Bucket S3 (e opcionalmente a flag ***use_lockfile*** para Lock), conforme visto na Aula 2 de IaC.

2. Pipeline de Integração Contínua (CI) & DevSecOps

Crie workflows (ex: GitHub Actions) para cada um dos 5 microsserviços. O pipeline deve rodar a cada Pull Request e Push na Main.

O pipeline deve conter os seguintes estágios (Jobs):

1. **Build & Unit Test:** Compilar o código e rodar testes unitários (se houver).
2. **Linter/Static Analysis:** Rodar ferramentas de linting (ex: golangci-lint para Go, pylint/flake8 para Python).
3. **Security Scan (SAST & SCA):**
   - **SCA (Software Composition Analysis):** Verificar vulnerabilidades nas dependências (ex: usar Trivy em modo fs ou OWASP Dependency Check).
   - **SAST (Static Application Security Testing):** Verificar vulnerabilidades no código fonte (ex: SonarCloud gratuito ou gosec/bandit).
   - ***Regra de Bloqueio:*** Se uma vulnerabilidade CRÍTICA for encontrada, o pipeline deve falhar e não prosseguir.
4. **Docker Build & Push:**
   - Construir a imagem Docker.
   - Rodar um scan de vulnerabilidades na imagem (Container Scan com Trivy).
   - Logar no AWS ECR.
   - Enviar a imagem para o ECR com a tag do commit hash (ex: v1.0.0-a1b2c3d).

3. Entrega Contínua (CD) & GitOps

Para o deploy, abandonaremos o push direto via CI. Vamos adotar o GitOps.

1. **Repositório de GitOps:** Crie um repositório separado (ou uma pasta separada no monorepo) contendo apenas os manifestos Kubernetes (YAMLs) ou Helm Charts das aplicações.

2. **Instalação do ArgoCD:** Instale o ArgoCD no seu cluster EKS (pode usar Helm ou Terraform com provider helm/kubectl).
3. **Atualização Automática:**
   - Ao final do pipeline de CI (passo anterior), adicione um passo que atualiza a tag da imagem no repositório de GitOps (alterando o arquivo deployment.yaml com a nova tag da imagem gerada).
4. **Sync:** Configure o ArgoCD para monitorar esse repositório e sincronizar automaticamente as mudanças para o cluster EKS.
   - Mostre a interface do ArgoCD gerenciando os 5 microsserviços.

ENTREGÁVEIS DA FASE 3

Vídeo de Demonstração (até 20 min):

- **IaC:** Mostre o `terraform plan` e o `terraform apply` rodando (ou o resultado final na AWS: VPCs, RDS, EKS criados via código).
- **Pipeline DevSecOps:** Faça uma alteração no código de um microsserviço (ex: insira um erro proposital ou uma dependência vulnerável) e mostre o pipeline **falhando** no passo de segurança. Depois corrija e mostre passando.
- **GitOps:** Mostre o pipeline atualizando a tag da imagem no repositório de GitOps.
- **ArgoCD:** Mostre o ArgoCD detectando a mudança e sincronizando a nova versão no cluster automaticamente.

Código Fonte no Repositório:

- Todo o código Terraform (bem estruturado e componentizado).
- Arquivos de Workflow (.yaml) do GitHub Actions (ou ferramenta similar) com os passos de DevSecOps implementados.
- Manifestos Kubernetes ajustados para o GitOps.

Relatório de Entrega (.PDF ou .txt):

- Nomes dos participantes.
- Link da documentação e do vídeo.
- Breve resumo dos desafios encontrados e decisões tomadas.
- Print da estimativa de custos da AWS.

Lembrando que qualquer dúvida, podem nos chamar no Discord.

Boa sorte!

POS TECH